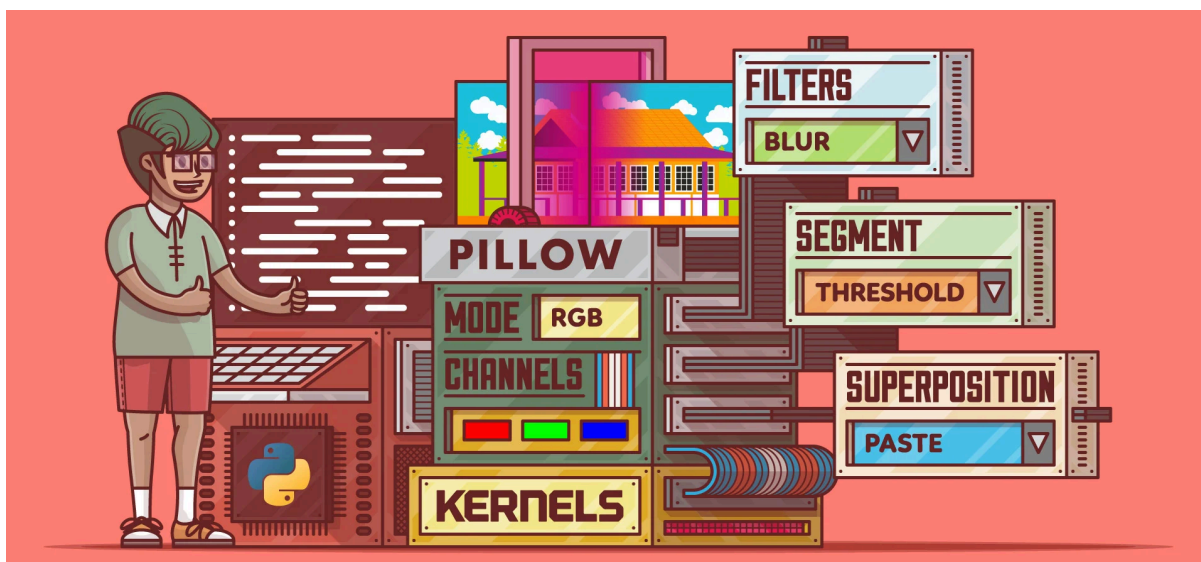


Project Hands V3



Index

ARTIFICIAL VISION PROJECT.....	1
Introduction.....	3
Possible improvements.....	3
Current issues.....	3
Possible applications.....	4
Libraries used.....	4
Code explanation.....	5
Initial setup.....	5
Hand Configuration.....	5
Landmarks on the hand.....	6
Main loop.....	6
Pre-processing.....	6
Hand detection and landmarks extraction.....	7
Coordinates on screen.....	8
Viewing and exiting.....	8
Benchmarks and response times.....	9
Event Diagram.....	9

Introduction

In this project I developed a mouse cursor control system through hand gesture recognition, using the MediaPipe library for tracking and PyAutoGUI for interaction with the operating system. The main goal was to explore the potential of computer vision, creating a natural and contactless interface that allows physical control of the computer and through simple hand movements it is possible to control the latter. The system was designed to be used in real time and without additional hardware, simply using a standard webcam.

The operation is as follows:

With the wrist closed the cursor moves, only the index finger raised corresponds to the left click, only the middle finger raised corresponds to the right click. To drag just move the cursor on what you want to select and keep the index finger raised and move the cursor to drag, to release just lower the index finger.

Possible improvements

Currently, the system works fairly well in optimal lighting conditions and with simple backgrounds, but there is still room for improvement. One of the main directions to work on is the integration of motion stabilization filters, in order to make cursor control smoother. Another possible evolution of the project could be the addition of support for more complex gestures, such as pinch-to-zoom or swipe (perhaps introducing the use of the left hand, although it could cause a more excessive use of computational complexity), as well as the ability to configure custom gestures. Furthermore, an automatic calibration could be implemented that adapts the system behavior based on the environment and the user's position.

Current issues

One of the main limitations concerns the system's performance in low-light environments or with backgrounds with many details. In these conditions, the tracking algorithm is less precise and can cause unstable behavior or jerky cursor movements. This is due to the difficulty for the model to correctly distinguish the hand from the context. Furthermore, even small delays in gesture detection can compromise the user experience, making it necessary to improve frame management and video stream optimization.

Possible applications

- The ability to control the computer without a physical mouse, which is particularly useful in environments where hygiene is paramount, such as healthcare facilities or laboratories.
- Accessibility for people with mobility difficulties.
- Integration with video games offering a more immersive user experience (and also it is a system very familiar to VR around, clearly much less precise and in 2D instead of 3D).
- It could be used in the educational field in Public Administration (offices, schools, etc.) for presentations or educational contexts to navigate between slides or contents dynamically and without contact.

Libraries used

cv2OpenCV is used to handle webcam and image processing.

media pipeIt is a Google library for tracking body, face, hands, etc.

pyautoguiIt is used to simulate mouse/keyboard input.

numpyI use it to interpolate coordinates (mapping between video and screen).

timeI use it to manage the cooldown between two clicks.

Code explanation

Initial setup

```
pyautogui.FAILSAFE = False
cap = cv2.VideoCapture(1)
cap.set(3,640)
cap.set(4,480)
```

Turn off PyAutoGUI failsafe (which otherwise freezes everything if the mouse goes to the top left corner). Then open the webcam (with index 1) and sets the size of the webcam window.

Hand configuration

```
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    max_num_hands=1,
    min_detection_confidence=0.8,
    min_tracking_confidence=0.8
)
mp_draw = mp.solutions.drawing_utils
```

First load the hand tracking module, the value `max_num_hands=1` consider only one hand for simplicity, while `min_detection_confidence` and `min_tracking_confidence` They are high on purpose to have greater precision. Finally `mp_draw` It is used to draw the hand on the frame.

Landmarks on the hand

```
def fingers_up(lm_list):  
    tips = [8, 12, 16, 20] # Index, Middle, Ring, Little up = []  
  
    for tip in tips:  
        up.append(lm_list[tip][1] < lm_list[tip - 2][1])  
    return up
```

It takes the list of landmarks (i.e. coordinates) of the fingers and compares the y-coordinate of the tip with that of the base for each finger to understand if it is raised.

Main loop

```
while True:  
    success, frame = cap.read() if  
    not success:  
        continue
```

Capture a frame from the webcam, if it fails it re-enters the loop.

Pre-processing

```
frame = cv2.flip(frame, 1)  
frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)  
results = hands.process(frame_rgb)
```

Flips the image horizontally to get a mirror effect, converts from BGR to RGB (because MediaPipe uses RGB) and finally processes the frame with MediaPipe to find the hand.

Hand detection and landmarks extraction

```
if results.multi_hand_landmarks:
    for hand_landmarks, handedness in zip(results.multi_hand_landmarks,
    results.multi_handedness):
        hand_label = handedness.classification[0].label
```

if results.multi_hand_landmarks checks if MediaPipe found at least one hand in the frame.

results is the object returned by hands.process(frame_rgb).

multi_hand_landmarks is a list of detected hands. If it is not, means that no hand was found, so it does not enter the block.

for hand_landmarks, handedness in zip(results.multi_hand_landmarks, results.multi_handedness) iterates over the found hands and their information (position + right/left). multi_hand_landmarks contains landmarks (i.e. points of the hand such as thumb, index finger, etc.). multi_handedness contains the hand classification (whether it is Right or Left). zip() It is used to take the landmarks and classification of each hand in parallel.

hand_label = handedness.classification[0].label extracts the hand label, i.e. whether it is Right or Left handedness is an object with a list called classification, classification[0].label contains the string Right or Left.

This allows me to do different actions only if it's the right hand, for example to not confuse the hands if there are two in the frame.

So, to summarize, this part checks if there is at least one hand in the video, and then for each hand it extrapolates its landmarks and understands if it is the right or the left, saving it in the variable hand_label.

Coordinates on screen

```
lm_list = [(int(lm.x * frame.shape[1]), int(lm.y * frame.shape[0])) for lm in
hand_landmarks.landmark]
mp_draw.draw_landmarks(frame, hand_landmarks,
mp_hands.HAND_CONNECTIONS)

up = fingers_up(lm_list)
wrist = lm_list[0]
x_screen = np.interp(wrist[0], (0, frame.shape[1]), (0, screen_width))
y_screen = np.interp(wrist[1], (0, frame.shape[0]), (0, screen_height))

dx = x_screen - prev_x
dy = y_screen - prev_y
```

Calculate the position of each landmark, then draw the hand on the screen and use the wrist (landmark 0) to control the mouse, then map the wrist coordinates from the frame to the desktop.

Viewing and exiting

```
cv2.imshow("Hand Controller", frame) if
cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
```

Shows the window with the video and the traced hand. Closes everything if you pressq.

Benchmarks and response times

For a low quality webcam with maximum 25 FPS varies between 6 and 7 in application, the time for each frame is approximately 0.1524 seconds.

Event Diagram

